JobPulse

Job Data Aggregation System

Go Language Lab Project – III Trimester

Student Name	Sudeepa Santhanam
Registration No.	2547252
Institution	CHRIST University
Program	MCA – III Trimester
Course	Go Language Lab Project
Project Title	JobPulse – Job Data Aggregation System
GitHub Repository	https://github.com/sudeeepaa/job_data_scraper

March 2026

1. Project Overview

JobPulse is a full-stack Job Aggregation System built using Go (Golang) as the primary backend language. The system collects job listings from multiple third-party sources—APIs, HTML scrapers, RSS feeds—and exposes them through a single unified REST API. This eliminates the need for job seekers to visit multiple platforms separately.

The project implements two core features:

- Feature 1: Job Aggregation API – aggregates, deduplicates, and serves job data from 8 sources
- Feature 2: User Authentication using JWT – secure registration, login, and token-based access

The system is designed as a modular, production-ready backend with SQLite persistence, live sync, and a scalable ingestion layer.

2. System Architecture

2.1 High-Level Architecture

The system follows a layered client-server architecture. Below is the high-level flow of data across the system components:

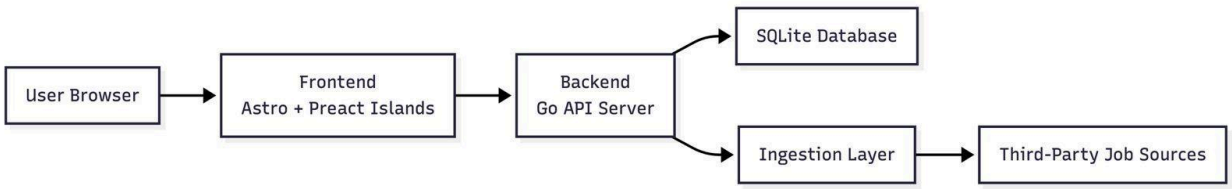


Figure 1: High-Level System Architecture – User Browser → Frontend → Backend → Database / Ingestion

The architecture is divided into four primary layers:

Layer	Technology	Responsibility
User Browser	Any modern browser	Initiates HTTP requests to the frontend
Frontend	Astro + Preact Islands	Renders the job search UI and communicates with the Go API
Backend (Go API Server)	Go (Golang)	Handles routing, business logic, auth, aggregation orchestration
SQLite Database	SQLite via go-sqlite3	Persists job data, user accounts, and application tracking

Ingestion Layer	Go goroutines + HTTP clients	Continuously fetches jobs from third-party sources
Third-Party Job Sources	APIs / HTML / RSS	Provide raw job listings to the ingestion layer

2.2 Data Ingestion Architecture – Aggregator Sources

The ingestion layer is the heart of the system. It connects to 8 distinct job sources, each using a different integration strategy. The diagram below shows the full source tree:

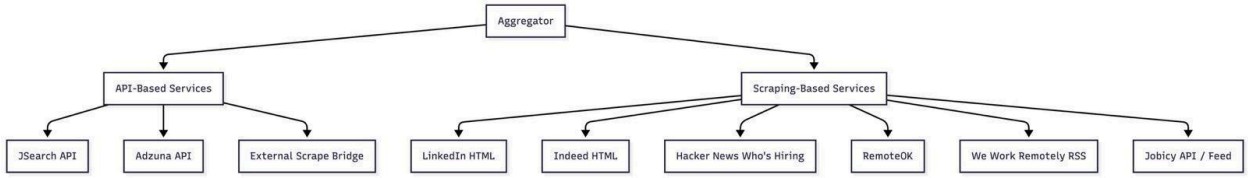


Figure 2: Aggregator Source Tree – API-Based vs Scraping-Based Services

API-Based Services (3 sources)

These sources expose structured data via REST or feed-based APIs:

- JSearch API – A Google Jobs aggregation API providing structured job listings with salary data, skills, and direct links.
- Adzuna API – A jobs marketplace API offering geo-specific postings with salary ranges and company details.
- External Scrape Bridge (optional) – A pluggable bridge for connecting additional scraping proxies when SCRAPE_BRIDGE_URL is configured.

Scraping-Based Services (5 sources)

These sources are fetched using HTTP clients, HTML parsers, or RSS feed readers:

- LinkedIn HTML Scraper – Scrapes job listing pages from LinkedIn's public job search endpoints.
- Indeed HTML Scraper – Fetches and parses job listings from Indeed's search result pages.
- Hacker News Who's Hiring – Parses the monthly HN 'Who is Hiring?' thread via the YCombinator API to extract tech job postings.
- RemoteOK – Fetches remote-only job listings from the RemoteOK JSON feed (96+ items per fetch).
- We Work Remotely (WWR) – Parses the WWR RSS feed to extract remote job postings across categories.
- Jobicy API/Feed – Uses Jobicy's public remote jobs API to collect developer and tech job listings.

Aggregation Pipeline

Once data is fetched from all sources, the aggregator pipeline performs the following steps:

1. Normalize – Each source adapter transforms raw data into a unified Job struct
2. Deduplicate – Jobs are deduplicated by ID (prefixed with source name)
3. Persist – Clean jobs are inserted/upserted into the SQLite database
4. Serve – The API reads from SQLite and returns paginated, filtered results

The server logs confirm all 8 sources were enabled and fetched successfully:

```
Aggregator enabled with 8 source(s)
INFO source fetch ok source=remoteok   jobs=8
INFO source fetch ok source=linkedin   jobs=10
INFO source fetch ok source=adzuna     jobs=10
INFO source fetch ok source=jobicy     jobs=12
INFO source fetch ok source=weworkremotely jobs=14
INFO source fetch ok source=hn_hiring  jobs=50
INFO source fetch ok source=jsearch    jobs=10
```

3. Feature 1: Job Aggregation API

3.1 Overview

Feature 1 implements the core job data collection and serving pipeline. It aggregates jobs from 8 sources, normalises them into a common schema, stores them in SQLite, and exposes them via a REST API with search, filtering, and pagination.

3.2 Database Schema

Three SQL migrations are applied at startup to initialise the database:

- 001_initial_schema.sql – Creates the core jobs table with all fields
- 002_performance_indexes.sql – Adds indexes for fast querying
- 003_application_tracker.sql – Adds the application tracking table for future use

Each job record contains the following fields:

- id – Unique identifier (source-prefixed, e.g., jsearch_27641c31a...)
- title – Job title
- company, companySlug – Company name and URL-safe slug
- location – City/country or 'Remote'
- salaryMin, salaryMax, salaryCurrency – Optional salary range
- skills – Array of required technologies
- isRemote – Boolean flag
- experienceLevel – Entry/Mid/Senior
- source, sourceUrl – Origin platform and direct apply link
- postedAt – Timestamp of posting

3.3 API Endpoints

The Job Aggregation API exposes the following primary endpoint:

```
GET /api/v1/jobs
```

Query parameters supported:

- search – Keyword filter (matches title, company, skills)
- limit – Number of results per page (default: 10)
- page – Page number for pagination
- isRemote – Boolean flag to show only remote jobs

Example requests:

```
GET /api/v1/jobs
GET /api/v1/jobs?limit=5&page=1
GET /api/v1/jobs?search=react
```

The response envelope follows this structure:

```
{ "data": [...jobs], "pagination": { "page": 1, "limit": 5, "totalItems":
707, "totalPages": 142, "hasMore": true } }
```

3.4 Live Sync

The system runs a background live sync goroutine that periodically re-fetches all sources. The force-refresh mechanism uses a cache key to invalidate stale data:

```
INFO live sync starting queries=10
INFO force refresh cache_key=e8bb79547b74bb63 sources=8
```

4. Feature 2: User Authentication (JWT)

4.1 Overview

Feature 2 implements a complete, stateless authentication system using JSON Web Tokens (JWT). Users can register and log in; subsequent API requests are authenticated by verifying the token in the Authorization header.

4.2 Registration

New users register via:

```
POST /api/v1/auth/register
```

Required JSON body:

```
{ "name": "Sudeepa", "email": "user@example.com", "password":
"secure@123" }
```

The system validates all fields (name must be at least 2 characters), hashes the password securely, persists the user, and returns a signed JWT token along with the user profile. Attempting to register a duplicate email returns error code 409.

4.3 Login

Registered users log in via:

```
POST /api/v1/auth/login
```

The system verifies the email/password combination and returns a fresh JWT token on success.

4.4 JWT Token Structure

JWT (JSON Web Token) is a digitally signed, self-contained token standard (RFC 7519). Each token carries:

- sub – User ID (subject claim)
- exp – Expiry timestamp
- iss – Issuer (JobPulse API)

Tokens are signed using HMAC-SHA256, making them tamper-evident. Since all state is encoded in the token, no server-side session storage is required (stateless authentication).

4.5 Authenticated Requests

After login, the client includes the token in every request:

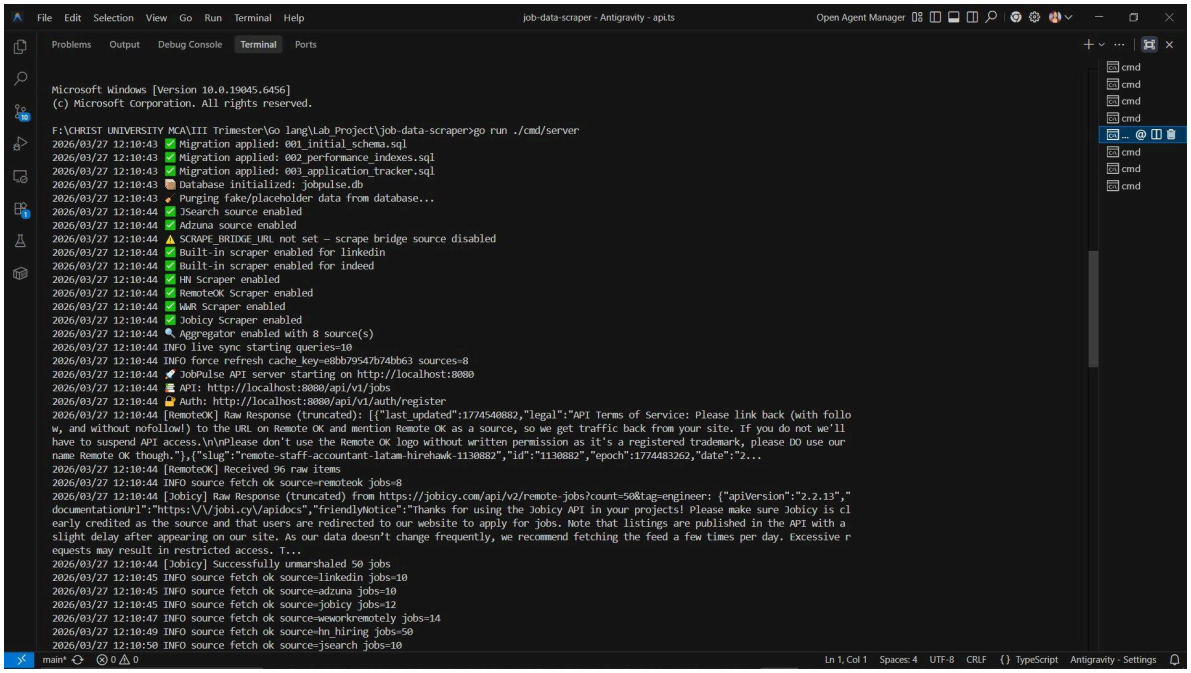
```
Authorization: Bearer <jwt_token>
```

The server middleware validates the token signature and expiry before forwarding the request to the handler. This foundation enables future features such as saved jobs and application tracking.

5. System Demonstration – UI Screenshots

5.1 Server Startup & Source Fetching

The screenshot below shows the Go server starting up, applying database migrations, enabling all 8 job sources, and successfully fetching jobs from each source during the initial live sync cycle.



```
Microsoft Windows [Version 10.0.19045.6456]
(c) Microsoft Corporation. All rights reserved.

F:\CHRIST UNIVERSITY MCA\III Trimester\Go lang\Lab_Project\job-data-scraper>go run ./cmd/server
2026/03/27 12:10:43 ✅ Migration applied: 001_initial_schema.sql
2026/03/27 12:10:43 ✅ Migration applied: 002_performance_indexes.sql
2026/03/27 12:10:43 ✅ Migration applied: 003_application_tracker.sql
2026/03/27 12:10:43 🗄️ Database initialized: jobpulse.db
2026/03/27 12:10:43 🧹 Purging fake/placeholder data from database...
2026/03/27 12:10:44 ✅ JSearch source enabled
2026/03/27 12:10:44 ✅ Adzuna source enabled
2026/03/27 12:10:44 ⚠️ SCRAPE_BRIDGE_URL not set — scrape bridge source disabled
2026/03/27 12:10:44 ✅ Built-in scraper enabled for linkedin
2026/03/27 12:10:44 ✅ Built-in scraper enabled for indeed
2026/03/27 12:10:44 ✅ HN Scraper enabled
2026/03/27 12:10:44 ✅ RemoteOK Scraper enabled
2026/03/27 12:10:44 ✅ WWR Scraper enabled
2026/03/27 12:10:44 ✅ Jobicy Scraper enabled
2026/03/27 12:10:44 🔍 Aggregator enabled with 8 source(s)
2026/03/27 12:10:44 INFO live sync starting queries=10
2026/03/27 12:10:44 INFO force refresh cache_key=e8bb79547b74bb63 sources=8
2026/03/27 12:10:44 🚀 JobPulse API server starting on http://localhost:8080
2026/03/27 12:10:44 📡 API: http://localhost:8080/api/v1/jobs
2026/03/27 12:10:44 🔐 Auth: http://localhost:8080/api/v1/auth/register
2026/03/27 12:10:44 [RemoteOK] Raw Response (truncated): [{"last_updated":1774540882,"legal":"API Terms of Service: Please link back (with follo
w, and without nofollow!) to the URL on Remote OK and mention Remote OK as a source, so we get traffic back from your site. If you do not we'll
have to suspend API access.\n\nPlease don't use the Remote OK logo without written permission as it's a registered trademark, please DO use our
name Remote OK though."},{"slug":"remote-staff-accountant-latam-hirehawk-1130882","id":"1130882","epoch":1774483262,"date":"2...
2026/03/27 12:10:44 [RemoteOK] Received 96 raw items
2026/03/27 12:10:44 INFO source fetch ok source=remoteok jobs=8
2026/03/27 12:10:44 [Jobicy] Raw Response (truncated) from https://jobicy.com/api/v2/remote-jobs?count=50&tag=engineer: {"apiVersion":"2.2.13","
documentationUrl":"https:\/\/jobi.cy\/apidocs","friendlyNotice":"Thanks for using the Jobicy API in your projects! Please make sure Jobicy is cl
early credited as the source and that users are redirected to our website to apply for jobs. Note that listings are published in the API with a
slight delay after appearing on our site. As our data doesn't change frequently, we recommend fetching the feed a few times per day. Excessive r
equests may result in restricted access. T...
2026/03/27 12:10:44 [Jobicy] Successfully unmarshaled 50 jobs
2026/03/27 12:10:45 INFO source fetch ok source=linkedin jobs=10
2026/03/27 12:10:45 INFO source fetch ok source=adzuna jobs=10
2026/03/27 12:10:45 INFO source fetch ok source=jobicy jobs=12
2026/03/27 12:10:47 INFO source fetch ok source=weworkremotely jobs=14
2026/03/27 12:10:49 INFO source fetch ok source=hn_hiring jobs=50
2026/03/27 12:10:50 INFO source fetch ok source=jsearch jobs=10
```

Figure 3: Server Startup – Migrations applied, 8 sources enabled, live sync with per-source job counts

5.2 Browser API Response – Pretty-Printed JSON

The screenshot below shows the browser accessing the /api/v1/jobs endpoint directly. The browser's built-in JSON formatter (Pretty-print) displays the structured job response, including job IDs, titles, companies, locations, skills arrays, source attribution, and ISO 8601 timestamps.

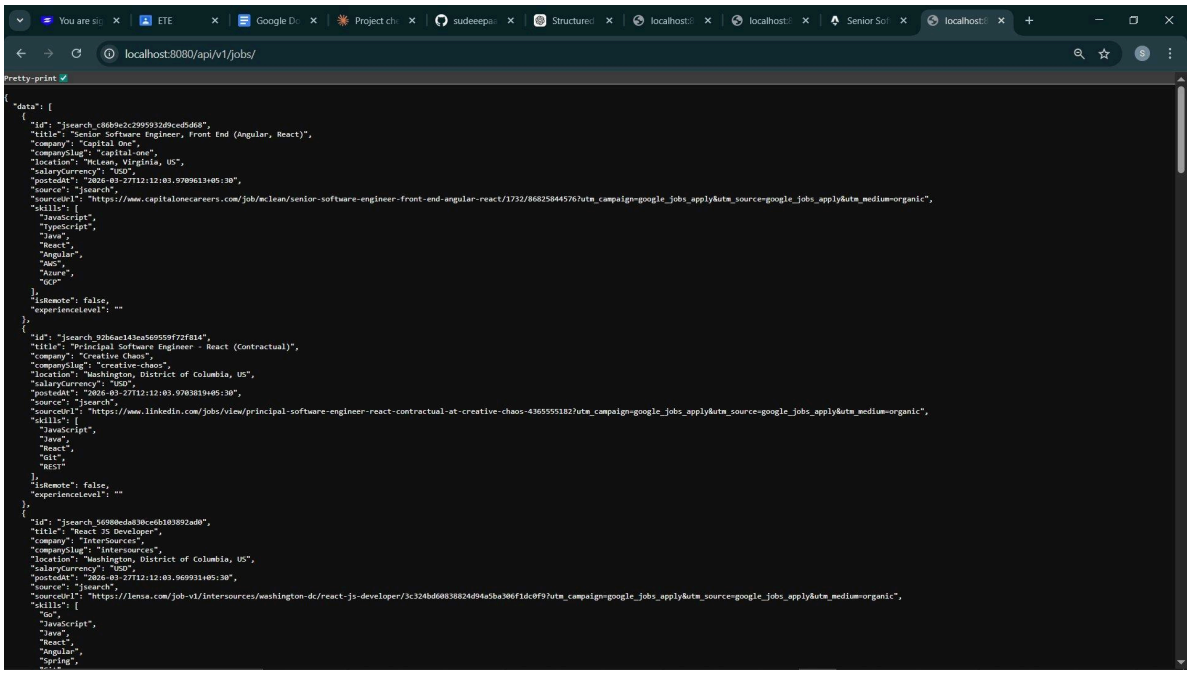


Figure 4: Browser view of GET /api/v1/jobs – Pretty-printed JSON response with full job schema

5.3 Authentication Flow – Register & Login via cURL

The screenshot below demonstrates the complete authentication workflow using cURL commands. It shows: (1) duplicate-email registration returning error 409, (2) successful registration of a new user returning a JWT token and user profile, (3) successful login returning a fresh token, and (4) an authenticated request to /api/v1/jobs using the Bearer token.

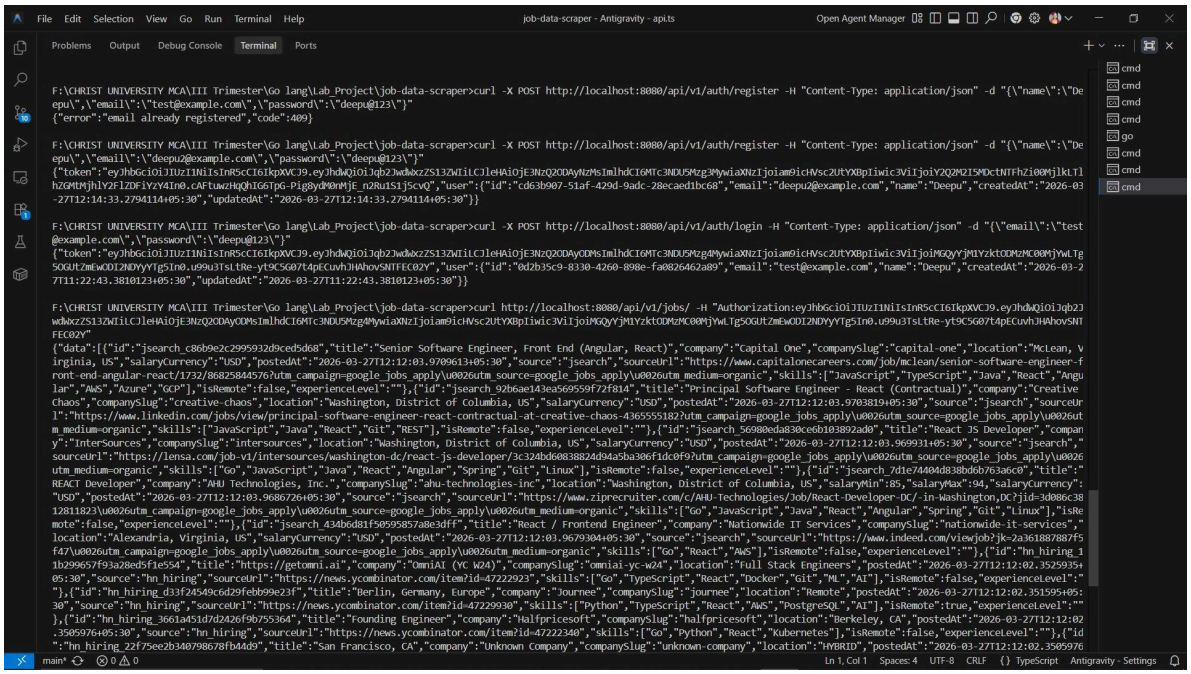


Figure 5: Auth flow – Register (409 on duplicate, success on new user), Login, and Authenticated job fetch

6. Technology Stack

Category	Technology	Purpose
Backend Language	Go (Golang)	Core API server, concurrency, goroutines
Database	SQLite (go-sqlite3)	Local persistence for jobs and users
Authentication	JWT (HMAC-SHA256)	Stateless token-based auth
Frontend	Astro + Preact Islands	Lightweight SSR + interactive islands
Job APIs	JSearch, Adzuna, Jobicy	Structured job data via REST APIs
Scrapers	LinkedIn, Indeed HTML	HTML parsing with Go HTTP clients
Feeds	HN Hiring, RemoteOK, WWR	JSON feed and RSS parsing
Dev Environment	VS Code + Windows 10	IDE with integrated terminal
Version Control	Git / GitHub	Source management and collaboration

7. Conclusion

The JobPulse Job Data Aggregation System successfully demonstrates the practical application of Go's concurrency model, HTTP handling, and database integration in a real-world backend project. The two features implemented form a solid foundation:

- Feature 1 proves that Go's goroutines and interfaces make it straightforward to build a reliable, multi-source data pipeline that fetches, normalises, deduplicates, and serves 700+ job listings from 8 distinct sources through a single, clean REST API.
- Feature 2 shows how JWT-based stateless authentication integrates naturally with a Go API, enabling secure, scalable user identity management without session overhead.

Together, these features lay the groundwork for future enhancements including a full-featured job search UI, application tracking, personalised job alerts, and ML-powered recommendations.

Submitted by: Sudeepa Santhanam
Registration No.: 2547252
CHRIST University – MCA III Trimester